

BUILD A LARGE LANGUAGE MODEL

Capítulo 1 - Understanding large language models (Entendendo grandes modelos de linguagem)

O que caracteriza um Large Language Model?

Um LLM é uma rede neural profunda treinada com grandes quantidades de dados de texto. Ele possui uma grande quantidade de parâmetros, que são ajustados durante o treinamento para que o modelo aprenda padrões da linguagem.

Uma característica fundamental dos LLMs é o treinamento para prever o próximo token/palavra de uma sequência. Essa tarefa é feita por meio de aprendizado auto-supervisionado, permitindo utilizar grandes volumes de textos sem a necessidade de humanos rotularem cada exemplo.

O termo "Large" está relacionado principalmente à grande escala dos modelos, quantidade de parâmetros e volume/diversidade dos dados utilizados no treinamento.

Basicamente são sistemas treinados com bilhões de dados de texto para entender, resumir, traduzir e gerar linguagem humana de forma natural. Acredito que a maior característica de LLMs é sua capacidade de gerar e “entender” a linguagem humana a partir de grandes quantidades de texto usadas durante o treinamento. É importante pontuar, que muitos podem achar que a maior característica de LLM é a arquitetura transformer, mas não estaria correto afirmar isso, visto que, existem LLMs que não utilizam essa base de arquitetura, existem modelos que são baseado em arquiteturas convolucionais (processar dados dispostos em formato de grade, como imagens através de convolução que é uma operação matemática).

Qual a diferença entre IA, Machine Learning e Deep Learning?

IA: é o campo mais amplo. Busca criar sistemas capazes de realizar tarefas que normalmente exigiriam algum tipo de inteligência humana, como tomar decisões, reconhecer padrões e compreender linguagem. Pode utilizar regras, sistemas especialistas, lógica simbólica, algoritmos genéticos, machine learning etc.

Machine Learning: é uma área da IA em que os algoritmos aprendem padrões a partir de dados, permitindo realizar previsões ou tomar decisões sem que todas as regras precisem ser programadas manualmente. No ML tradicional, é comum que especialistas precisem selecionar ou extrair características relevantes dos dados.

Deep Learning: é um subconjunto de Machine Learning que utiliza redes neurais profundas, com várias camadas, para aprender representações e padrões complexos diretamente dos dados. Diferentemente de muitos métodos tradicionais de ML, ele consegue aprender automaticamente características relevantes, reduzindo a necessidade de engenharia manual de características.

Como os modelos GPT estão organizados?

Os GPTs (Generative Pre-trained Transformers) utilizam a arquitetura Transformer, mas focam principalmente no submódulo decodificador da arquitetura original. A Transformer original possui encoder (codificador) que recebe o texto de entrada e o transforma em representações numéricas e o decoder (decodificador) que utiliza essas representações para gerar o texto de saída.

O GPT utiliza a parte decoder e é um modelo autorregressivo (regressão estatística, prever o futuro baseado no passado). Isso significa que ele gera o texto de forma sequencial, prevê um próximo token com base nos tokens anteriores e depois utiliza sua própria saída como parte da entrada para a próxima previsão.

Por isso, o GPT é especialmente adequado para geração de texto, podendo realizar tarefas como escrita, tradução, geração de código e outras tarefas por meio de zero-shot e few-shot learning.

Quais são as principais etapas do treinamento de um LLM?

O processo geral pode ser dividido principalmente em duas etapas: Pré-treinamento (pretraining), onde o modelo é treinado utilizando um grande e diverso conjunto de textos, geralmente sem rótulos humanos. Ele aprende por aprendizado auto-supervisionado, tentando prever o próximo token da sequência. Por exemplo, "O gato está sobre o...". O modelo aprende a prever uma continuação provável, como "tapete". Esse processo permite que o modelo desenvolva uma representação ampla da linguagem e forme um modelo de fundação.

Fine-tuning (afinação), depois do pré-treinamento, o modelo pode ser ajustado utilizando um conjunto de dados menor e rotulado, direcionado a determinadas tarefas. Instruction fine-tuning que é o treinamento com pares de instruções e respostas e classification fine-tuning que é o treinamento com textos associados a categorias, como "spam" e "não spam".

Assim, o pré-treinamento proporciona uma capacidade geral de linguagem, enquanto o fine-tuning pode especializar o modelo para determinadas tarefas.

Qual é o fluxo geral de funcionamento de um modelo de linguagem?

Fluxo em resumo: Texto de entrada → tokens → representações numéricas → processamento pela Transformer → previsão do próximo token → novo token → repetição → texto gerado

Primeiro, o texto é dividido em tokens. Esses tokens são transformados em representações numéricas que podem ser processadas pela rede neural. Dentro da Transformer, o mecanismo de autoatenção (self-attention) permite que o modelo considere a relação entre diferentes tokens da sequência e determine quais informações são mais relevantes para fazer a previsão.

No caso de um GPT, o modelo então prevê o próximo token. Esse token é acrescentado à sequência, e o modelo faz uma nova previsão considerando os tokens anteriores. Esse processo continua até que o modelo produza uma sequência de texto completa.

Anotações:

LLMs:

- LLMs são treinadas com grandes quantidades de dados de texto, permitindo capturar contextos de informação mais profundos e sutilezas da linguagem humana comparado com abordagens anteriores (NLP).
- LLM é uma rede neural feita para entender, gerar e responder textos com aspecto humano. São redes neurais profundas treinadas com gigantes quantidades de dados de texto, dados esses muitas vezes advindos de textos disponíveis publicamente na internet. O “Large” refere-se ao tamanho do modelo de parâmetros e imensos datasets em que são treinadas. Esses parâmetros são os “pesos ajustados” na rede que são otimizados durante treinamento para prever a próxima palavra na sequência. O next-word é sensível visto que aproveita a natureza sequencial da linguagem.
- LLMs são usadas para geração de textos “inéditos”, tradução automática, criação de ficção, artigos, códigos de programação, chatbots e assistentes virtuais. Os LLMs são inestimáveis para automatizar qualquer tarefa que envolve análise e geração de texto.
- LLMs feitas para tarefas e domínios específicos podem superar o desempenho de LLMs com propósito geral (como o Chat GPT que é feito para uma ampla gama de aplicações).
- LLMs para tarefas específicas oferece vantagens principalmente em privacidade de dados. Desenvolver LLMs menores permite implementação diretamente nos dispositivos dos clientes (laptops, celulares e etc). LLMs menores podem reduzir latência, diminuir custos de servidor e garantir autonomia do desenvolvedor.

NLP para DNN:

- NLP é uma área da IA que procura fazer computadores entenderem, interpretarem, gerarem e manipularem a linguagem humana, fazer a máquina trabalhar com textos e fala de criação humana. Mas como computadores trabalham bem com números eles podem ter dificuldades como ambiguidade das palavras, contexto, gírias e abreviações.
- Antes das redes neurais profundas o NLP era baseado principalmente em regras e estatísticas, mas tinha problemas visto que eram muitas regras e precisava de muita engenharia manual.
- Então vieram as DNN que mudaram as NLP visto que conseguiam aprender representações da linguagem automaticamente

IA, Machine learning e Deep learning:

- Deep learning é um subconjunto de machine learning e IA mas que foca em redes neurais.

- IA abrange um campo maior de criar máquinas que podem realizar tarefas que requerem inteligência humana, incluindo entender linguagens, reconhecer padrões, tomar decisões, ela inclui subáreas como machine learning e deep learning.
- Deep learning abrange áreas das LLMs e GenAI, ela é basicamente machine learning com redes neurais. Ela é uma subárea de machine learning, focada em utilizar redes neurais com 3 ou mais camadas (deep neural networks) para modelar padrões complexos em dados. Ela não precisa de extração manual de características, entretanto assim como o aprendizado de máquina tradicional, ela requer coleção de rótulos que definem o que é algo e o que não é algo, que devem ser juntados por especialistas ou usuários.
- Machine learning abrange áreas de Deep learning, ele é o algoritmo que aprende as regras automaticamente através de vários dados. Os algoritmos usados para implementar IA são o foco do machine learning. O aprendizado de máquina envolve criar algoritmos que podem aprender e fazer previsões ou decisões baseados em dados sem serem explicitamente programados. Ao minimizar o erro nas previsões em um dataset de treinamento, o modelo aprende a reconhecer padrões e características indicativas, por exemplo, do spam em e-mails. Em contraste com o deep learning, o machine learning tradicional requer extração manual de características, ou seja, humanos especialistas precisam identificar e selecionar as características mais relevantes para o modelo.
- IA é um sistema com inteligência do tipo humana. Embora esteja dominada por aprendizado de máquina e deep learning, ela inclui outras abordagens (sistema baseado em regras, algoritmos genéticos, sistemas especialistas, lógica difusa/fuzzy, raciocínio simbólico).

Arquitetura da LLM:

- Chama de transformer, permite que esses modelos prestem atenção seletiva para diferentes partes da entrada quando fazem previsões, o que faz com sejam proficientes em lidar com nuances e complexidades da linguagem humana.
- A maioria das LLMs modernas se baseia na arquitetura transformers, que é uma rede neural profunda.
- A transformer original foi desenvolvida para tradução de máquinas (do inglês para o alemão e francês). Ela possui 2 submódulos (codificador e decodificador). O codificador processa a entrada de texto e codifica em uma série de vetores ou representações numéricas. Já o decodificador pega esses vetores codificados e gera o texto de saída. Ambos consistem em muitas camadas conectadas pelo mecanismo de autoatenção.
- O mecanismo de autoatenção permite ao modelo ponderar a importância de diferentes palavras ou tokens em uma sequência uns em relação aos outros. Permite o modelo capturar dependências de longa distância e relações contextuais dentro dos dados de entrada, assim aumentando a habilidade de gerar saídas coerentes e de contexto relevante.

- Variantes dessa arquitetura, BERT e GPT models, construíram-se nesse conceito e adaptaram a arquitetura para diferentes tarefas. BERT foca no submódulo de codificar e o GPT no decodificar. Enquanto o GPT foi feito para tarefas generativas, o BERT é especializado em predição de palavras mascaradas onde o modelo predita palavras mascaradas ou escondidas em uma frase e esse tipo único de treinamentos permite o BERT ter forças em tarefas de classificação de textos, previsão de sentimentos e categorização de documentos. O GPT foi feito para tarefas que requerem geração de textos (tradução, escrita de código, escrita de ficção e etc), apesar de terem sido feitos para completar textos ele mostra uma versatilidade notável, visto que esse modelo se adaptam em tarefas de zero-shot e few-shot. O aprendizado de zero-shot refere-se à habilidade de generalizar para uma tarefa completamente inédita sem qualquer especificidade prévia como exemplo. Já o few-shot aprende com um número mínimo de exemplos que o usuário fornece como entrada.
- Importante lembrar que nem todos transformers são LLM visto que eles podem ser usados para visão computacional também. Assim como, nem todas LLMs são transformers visto que tem LLM baseada em arquiteturas recorrentes e convolucionais.
- Esses pré treinamento fazem desses modelos GPT incrivelmente versáteis para afinações futuras e tarefas subsequentes. A escala e diversidade dos treinamentos de dataset permite os modelos performarem bem em diversas tarefas, incluindo linguagem, semântica e contexto.
- Pré treinar LLMs requer muitos recursos e muito dinheiro. Mas, muitos pré-treinamentos de LLMs estão disponíveis como modelos de código aberto e podem ser usados com ferramentas de propósito geral para escrever, extrair e editar textos que não são parte dos dados de treinamento. Assim como, podem ser afinadas para tarefas específicas com dataset menor e menos custo computacional.

Treinamento LLM:

- O processo geral de criar uma LLM inclui pré-treino e afinação. O pré treinamento refere-se a fase inicial onde um modelo LLM é treinado em um grande e diverso dataset para desenvolver um amplo entendimento da linguagem, ele serve como uma fonte fundamental que pode ser mais para frente refinada através da afinação (fine-tuning). A afinação é um processo onde o modelo é especificamente treinado em um dataset mais estreito e específico para tarefas particulares.
- O primeiro passo em criar uma LLM é treiná-la em grandes montes de dados de texto (raw text). Esses textos são apenas regulares sem serem transformados em informação. Filtros podem ser aplicados como remoção de formato de caracteres ou documentos em linguagens desconhecidas. Nessa fase (pretraining) os LLMs usam auto-supervisionado aprendizado, onde o modelo gera seus próprios rótulos dos dados de entrada. Criamos assim uma base ou modelo de fundação.

- Após esse treinamento, podemos treinar o LLM em dados rotulados (fine-tuning). As duas categorias mais populares de afinação são instruction fine-tuning e classification fine-tuning. Em instrução de afinação, o dataset rotulado consiste em instruções e pares de respostas, como uma consulta (query) para traduzir um texto acompanhado por textos corretamente traduzidos. Já em classificação de afinação, o dataset rotulado consiste em textos e rótulos de classe associados, por exemplo, e-mails associados como “spam” ou “não spam”.

Modelos GPT:

- O GPT foca no submódulo decodificar da arquitetura transformers. O GPT foi feito para tarefas que requerem geração de textos (tradução, escrita de código, escrita de ficção e etc), apesar de terem sido feitos para completar textos ele mostra uma versatilidade notável, visto que esse modelo se adaptam em tarefas de zero-shot e few-shot. O aprendizado de zero-shot refere-se à habilidade de generalizar para uma tarefa completamente inédita sem qualquer especificidade prévia como exemplo. Já o few-shot aprende com um número mínimo de exemplos que o usuário fornece como entrada.
- Esses modelos são completação de texto e podem ter outras tarefas como correção ortográfica, classificação ou tradução de linguagem. GPT são treinados em tarefas de predição da próxima palavra (sistema aprende a prever a próxima palavra em uma frase ao olhar para as palavras que vem antes, essa fundamentação pode ser aplicada para outras tarefas).
- Essa tarefa de predição da próxima palavra é uma forma de aprendizado auto supervisionado que é uma forma de auto rotular. Ou seja, não é preciso coletar os rótulos para o treinamento de dados, mas pode usar a estrutura do dado em si, usar a próxima palavra em um documento/frase como o alvo que o modelo deve prever. Permite assim, criarmos rótulos na hora, sendo possível usar grandes datasets de textos não rotulados para treinar as LLMs.
- Comparado com a arquitetura original de transformers o GPT é relativamente simples, é apenas a parte de decodificação, como ele geram textos ao prever texto uma palavra por vez, eles são considerados modelos autorregressivos, visto que, esse tipo de modelo incorpora suas saídas anteriores com entradas para predições futuras. Cada palavra nova é escolhida baseada na sequência que a precede, o que melhora a coerência do texto resultante.
- As arquiteturas GPT são significativamente maiores que as transformers originais. Por exemplo, a GPT-3 contém 96 camadas de transformers.
- GPT modelos também são capazes de performar tarefas de tradução. Essa habilidade de performar uma tarefa que o modelo não foi especificamente treinado para, é chamada de comportamento emergente, ou seja, essa habilidade emerge naturalmente como consequência da exposição de modelos para quantidades vastas de dados multilínguas em diversos contextos. Demonstra, assim, os benefícios e capacidades desses modelos de linguagem generativa e larga escala.

PyTorch:

- Biblioteca de aprendizado profundo.
- Lembrando IA é sobre criar um sistema computacional capaz de realizar tarefas que geralmente requerem inteligência humana (entendimento de linguagem, reconhecer padrões e tomar decisões).
- Lembrando Machine Learning é uma subárea da IA, foca em desenvolver e melhorar algoritmos de aprendizado. Permitir que computadores aprendam por dados e tomem decisões ou prevejam algo sem serem explicitamente programados para isso (algoritmos que identificam padrões, aprendem pelo histórico de dados e melhoram sua performance com o tempo).
- Lembrando Deep Learning é um Machine Learning com redes neurais com muitas camadas (profundas). Pode ser entendido como uma subárea de Machine Learning. Foca em treinar e aplicar redes neurais profundas.
- Aprendizado supervisionado é modelos que são treinados em dados já rotulados. Assim, quando treinado pode-se prever os rótulos dos novos dados.
- Tensores: generaliza vetores e matrizes para potencialmente dimensões maiores, são caracterizados pela sua ordem (número de dimensões). Escalar (só um número) é de ordem 0, vetor é ordem 1, matriz é ordem 2, e assim vai seguindo. Eles servem como containers de dados. Podem guardar multidimensionais dados em que cada dimensão é um recurso/característica. PyTorch cria, manipula e computa esses arrays. A Biblioteca de tensor funciona como biblioteca de arrays. A diferença é que PyTorch possui recursos que são importantes para deep learning (motor de diferenciação, suporte para GPU, simplificação de gradientes).
- Autograd: motor de diferenciação automático. O sistema de autograd tem funções que computam gradientes em grafos de computação dinâmica automaticamente. Grafo computacional permite expressar e visualizar expressões matemáticas. Em termos de deep learning o grafo mostra a sequência de cálculos necessários para computar a saída de uma rede neural.
- Ao realizar cálculos no PyTorch, o sistema constrói internamente um grafo computacional por padrão se um dos nós terminais têm o atributo "require_grad" definido como True. Isso é útil para calcular gradientes, eles são necessários para o treinamento de redes neurais por meio do algoritmo de retropropagação (regra da cadeia do cálculo aplicada a redes neurais)
- Backpropagation (Retropropagação): Um jeito de computar os gradientes de perda/loss envolve aplicar a regra da cadeia da direita para a esquerda (modelo automático reverso de diferenciação / backpropagation). Começa da camada de saída (ou de perda) e vai voltando através da rede até a camada de entrada, isso calcula o gradiente da função de perda em

relação a cada parâmetro (peso e bias/vieses), assim obtemos a informação de como nós atualizamos esses parâmetros durante o treinamento.

- Derivadas parciais: Elas medem a taxa que a função muda em relação a uma de suas variáveis.
- Gradientes: É um vetor que contém todas as derivadas parciais de uma função multivariada (mais de uma variável de entrada)
- A regra de cadeia é uma maneira de calcular gradientes de uma função de perda em relação aos parâmetros do modelo em um grafo de computação. Isso fornece as informações necessárias para atualizar cada parâmetro a fim de minimizar a função de perda, a qual serve como uma métrica para avaliar a performance do modelo utilizando um método como o gradiente descendente.
- Motor Autograd: Ele constrói um grafo computacional em segundo plano. Ele consegue fazer isso rastreando todas as operações realizadas nos tensores, em seguida chama a função de gradiente (grad) para calcular o gradiente da função de perda em relação ao parâmetro do modelo (ex: w_1)
- Biblioteca para implementar redes neurais de múltiplas camadas: Por exemplo, nela temos a camada de entrada (10 unidades de entrada), 1ª camada escondida (6 nós e 1 unidade bias), 2ª camada escondida (4 nós e 1 nó representando as unidades de bias) e a camada de saída (3 unidades de saída). Para implementar uma rede neural utiliza o [torch.nn](https://pytorch.org/docs/stable/nn.html). Module class (define a arquitetura de rede customizada). Esse Module faz com que seja mais fácil construir e treinar modelos, permite encapsular camadas e operações e vigiar os parâmetros do modelo. Define as camadas de rede no construtor `__init__` e especifica como as camadas interagem no método forward.
- Método Forward: descreve como os dados de entrada passam pela rede e juntam-se como um grafo de computação.
- Método Backward: usado durante treinamento para calcular os gradientes de função de perda. Não implementamos manualmente.
- Toydataset: conjunto de dados pequeno e artificial, só para testar o código.
- As classes sempre começam do 0. Se temos uma classe de 5 (0-4), a camada de saída da rede deve ser de 5 nós.
- Época de treinamento (training epoch): dataloader itera sobre o dataset, visitando cada exemplo exatamente 1 vez. É um hiperparâmetro, devemos escolher seu número também.
- Quando a loss atinge 0, isso significa que o modelo convergiu no conjunto de treinamento naquele batch específico.
- Taxa de aprendizado: É um hiperparâmetro, ou seja, é uma configuração ajustável que devemos realizar experimentos baseados em observações da loss. Idealmente, devemos escolher uma taxa em que a loss converge depois de um certo número de epochs.

- Cálculo de parâmetros: Deve-se observar as camadas, por exemplo camada 1 recebe 2 entradas e transforma em 30 valores, então $2 * 30 = 60$ pesos, cada valor (neurônio) possui 1 bias então são 30 bias e $60 + 30 = 90$. Faça isso para todas as camadas, soma e resulta na quantidade de parâmetros treináveis que existem nessa rede neural.
- Training set: Usada para treinar o modelo. Onde acontece o backward e o optimizer, ou seja, o modelo olha para os dados e altera os seus pesos.
- Validation set: Usada para escolher configurações do set. O modelo não é treinado, aqui utiliza o conjunto de validação para responder “Qual learning rate funciona melhor?”. Aqui temos o learning rate e a validation loss, qual taxa river a menor loss de validação, sabemos que é a melhor escolha. Ele pode ser testado várias vezes.
- Test set: Usada para fazer a avaliação final. Não testa-se várias vezes nesse set para não viciar a avaliação.
- Parâmetros são coisas que o modelo aprende, tipo o bias, pesos. Já os hiperparâmetros são configurações que você escolhe, tipo learning rate, batch, epochs, quantidade de camadas, neurônios e tipo de optimizer.
- `Model.train()`: modelo está treinando, `model.eval()`: agora modelo está avaliando.
- Algumas camadas da rede se comportam diferente dependendo se estamos treinando ou avaliando. Um exemplo é o dropout, em que durante treinamento ele pode “desligar” alguns neurônios a cada rodada, isso ajuda a rede a não depender demais de certos neurônios.
- Dispositivo (device): Onde as computações ocorrem e os dados residem. CPU e GPU são exemplos de dispositivos. O tensor reside em um dispositivo e suas operações são executadas no mesmo. Todos os tensores devem estar no mesmo dispositivo, não pode um estar na CPU e outro na GPU.
- Treinamento distribuído: é o conceito de dividir o modelo de treinamento em várias GPUs e máquinas. Apesar de ser possível treinar o modelo em uma única GPU/máquina, o processo pode demorar muito tempo. O tempo é reduzido ao distribuir o processo de treinamento em múltiplas máquinas, cada uma com múltiplas GPUs. Isso é crucial nas etapas de desenvolvimento do modelo. Usar o `DistributedDataParallel` é o jeito mais simples no PyTorch de acelerar o treinamento se múltiplas GPUs estão disponíveis.